

Rapport de Projet Frame

Dkhil Yosri
Azzi Moenes
Med Amine Aslouj



Form Connexion



The image shows a screenshot of a web application window titled "Connexion - Clinique Vétérinaire". The window has a standard Windows-style title bar with minimize, maximize, and close buttons. The main content area has a light gray background. At the top, the text "Clinique Vétérinaire" is displayed in a bold, black font. Below this, there are two input fields: "Login:" followed by a white text box, and "Mot de passe:" followed by a white text box. At the bottom of the form, there are two buttons: a green button labeled "Se connecter" and a red button labeled "Quitter".

1. Présentation

ce formulaire Il permet à un utilisateur (administrateur, secrétaire ou vétérinaire) de :

- saisir son login et mot de passe
- vérifier ses informations dans une base de données SQL Server
- accéder à une interface différente selon son rôle

Form Connexion

2. Importation des bibliothèques

System : fonctionnalités de base
Microsoft.Data.SqlClient : connexion à
SQL Server
System.Windows.Forms : création
d'interface graphique

```
using System;  
using Microsoft.Data.SqlClient;  
using System.Windows.Forms;
```

3.Initialisation

InitializeComponent() : initialise les Éléments
de l'interface graphique
connexionBD : prépare la connexion à la base de données

```
public Form1()  
{  
    InitializeComponent();  
    connexionBD = new ConnexionBD();  
}
```

4.btnConnexion_Click

Cette méthode gère la connexion de l'utilisateur. Elle vérifie d'abord que le login et le mot de passe sont remplis, puis interroge la base de données pour valider les informations. Si l'utilisateur existe, un message de bienvenue s'affiche et il est redirigé vers une interface selon son rôle. Sinon, un message d'erreur est affiché et les champs sont réinitialisés. Les erreurs de connexion sont également prises en charge.

```
private void btnConnexion_Click(object sender, EventArgs e)
{
    string login = txtLogin.Text.Trim();
    string password = txtMotDePasse.Text;

    if (string.IsNullOrEmpty(login) || string.IsNullOrEmpty(password))
    {
        MessageBox.Show("Veuillez saisir votre login et mot de passe",
            "Champs vides", MessageBoxButtons.OK, MessageBoxIcon.Warning);
        return;
    }

    using (SqlConnection conn = connexionBD.GetConnection())
    {
        try
        {
            conn.Open();
            string sql = "SELECT Id, Nom, Prenom, Role FROM Utilisateur WHERE Login=@Login AND MotDePasse=@MotDePasse";
            SqlCommand cmd = new SqlCommand(sql, conn);
            cmd.Parameters.AddWithValue("@Login", login);
            cmd.Parameters.AddWithValue("@MotDePasse", password);

            SqlDataReader reader = cmd.ExecuteReader();

            if (reader.Read())
            {
                string nom = reader["Nom"] != null ? reader["Nom"].ToString() : "";
                string prenom = reader["Prenom"] != null ? reader["Prenom"].ToString() : "";
                string role = reader["Role"] != null ? reader["Role"].ToString() : "";

                reader.Close();

                MessageBox.Show($"Bienvenue {prenom} {nom} !\nRôle: {role}",
                    "Connexion réussie", MessageBoxButtons.OK, MessageBoxIcon.Information);

                this.Hide();

                if (role == "Administrateur")
                {
                    FormAdministrateur adminForm = new FormAdministrateur(nom, prenom);
                    adminForm.Show();
                }
                else if (role == "Secrétaire")
                {
                    FormSecrétaire secretaireForm = new FormSecrétaire(nom, prenom);
                    secretaireForm.Show();
                }
                else if (role == "Vétérinaire")
                {
                    FormVeterinaire vetoForm = new FormVeterinaire(nom, prenom);
                    vetoForm.Show();
                }
            }
        }
        catch (Exception ex)
        {
            MessageBox.Show($"Erreur de connexion à la base de données:\n{ex.Message}",
                "Erreur", MessageBoxButtons.OK, MessageBoxIcon.Error);
        }
    }
}
```

```
    else if (role == "Veterinaire")
    {
        FormVeterinaire vetoForm = new FormVeterinaire(nom, prenom);
        vetoForm.Show();
        this.Hide();
    }
}
else
{
    MessageBox.Show("Login ou mot de passe incorrect",
        "Échec de connexion", MessageBoxButtons.OK, MessageBoxIcon.Error);
    txtLogin.Clear();
    txtMotDePasse.Clear();
    txtLogin.Focus();
}

reader.Close();
}
catch (Exception ex)
{
    MessageBox.Show($"Erreur de connexion à la base de données:\n{ex.Message}",
        "Erreur", MessageBoxButtons.OK, MessageBoxIcon.Error);
}
}
```

Form Admin

1. Présentation

La classe FormAdministrateur représente l'interface principale destinée aux utilisateurs ayant le rôle Administrateur dans l'application Clinique Vétérinaire.

Ce formulaire permet :
de gérer les utilisateurs
de gérer le stock
de consulter des statistiques globales
de se déconnecter du système

The screenshot shows a window titled "Administrateur - Clinique Vétérinaire". Inside, the header reads "Tableau de Bord Administrateur" followed by "Welcome, Admin!". On the left, there are four buttons: "Gestion des Utilisateurs" (light blue), "Gestion du Stock" (light blue), "Statistiques" (light blue), and "Déconnexion" (red). On the right, there is a large grey rectangular area. At the top of this area, there are two small white boxes labeled "Statistique" and "Nombre".

Form Admin

2. Importation des bibliothèques

System : fonctionnalités de base

SqlClient : interaction avec la base de données SQL Server

Windows.Forms : gestion de l'interface graphique

```
using System;
using Microsoft.Data.SqlClient;
using System.Windows.Forms;
```

3. Déclaration des attributs

nomUtilisateur : stocke le nom de l'administrateur

prenomUtilisateur : stocke le prénom

connexionBD : objet permettant d'accéder à la base de données

4. Constructeur du formulaire

Initialise les composants graphiques

Initialise la connexion à la base de données

Récupère les informations de l'utilisateur connecté

Affiche un message de bienvenue personnalisé

```
namespace CliniqueVeterinaire
{
    4 références
    public partial class FormAdministrateur : Form
    {
        private string nomUtilisateur;
        private string prenomUtilisateur;
        private ConnexionBD connexionBD;

        1 référence
        public FormAdministrateur(string nom, string prenom)
        {
            InitializeComponent();
            connexionBD = new ConnexionBD();
            nomUtilisateur = nom;
            prenomUtilisateur = prenom;
            lblWelcome.Text = $"Bienvenue, {prenom} {nom} !";
        }
    }
}
```

Form Admin

5. Méthode : ChargerStatistiques()

La méthode `ChargerStatistiques()` permet de récupérer des informations globales depuis la base de données. Elle établit une connexion SQL puis exécute trois requêtes pour compter le nombre d'utilisateurs, d'animaux et de rendez-vous. Les résultats obtenus sont ensuite affichés dans un tableau (`DataGridView`) afin de fournir une vue synthétique des données du système. En cas de problème lors de l'exécution, un message d'erreur est affiché pour informer l'utilisateur.

6. Gestion des utilisateurs

Les boutons du formulaire Administrateur permettent d'accéder aux principales fonctionnalités : gestion des utilisateurs, gestion du stock, affichage des statistiques et déconnexion sécurisée de l'application.

```
private void ChargerStatistiques()
{
    using (SqlConnection conn = connexionBD.GetConnection())
    {
        try
        {
            conn.Open();

            string sqlUsers = "SELECT COUNT(*) FROM Utilisateur";
            SqlCommand cmdUsers = new SqlCommand(sqlUsers, conn);
            int nbUsers = (int)cmdUsers.ExecuteScalar();

            string sqlAnimals = "SELECT COUNT(*) FROM Animal";
            SqlCommand cmdAnimals = new SqlCommand(sqlAnimals, conn);
            int nbAnimals = (int)cmdAnimals.ExecuteScalar();

            string sqlRdv = "SELECT COUNT(*) FROM RendezVous";
            SqlCommand cmdRdv = new SqlCommand(sqlRdv, conn);
            int nbRdv = (int)cmdRdv.ExecuteScalar();

            dgvDashboard.Rows.Clear();
            dgvDashboard.Rows.Add("Utilisateurs", nbUsers);
            dgvDashboard.Rows.Add("Animaux", nbAnimals);
            dgvDashboard.Rows.Add("Rendez-vous", nbRdv);
        }
        catch (Exception ex)
        {
            MessageBox.Show($"Erreur: {ex.Message}", "Erreur",
                MessageBoxButtons.OK, MessageBoxIcon.Error);
        }
    }
}
```

```
private void btnGestionUtilisateurs_Click(object sender, EventArgs e)
{
    FormGestionUtilisateurs form = new FormGestionUtilisateurs();
    form.ShowDialog();
}

1 référence
private void btnGestionStock_Click(object sender, EventArgs e)
{
    FormGestionStock form = new FormGestionStock();
    form.ShowDialog();
}

1 référence
private void btnStats_Click(object sender, EventArgs e)
{
    ChargerStatistiques();
    MessageBox.Show("Statistiques mises à jour",
        "Information", MessageBoxButtons.OK, MessageBoxIcon.Information);
}

1 référence
private void btnDeconnexion_Click(object sender, EventArgs e)
{
    DialogResult result = MessageBox.Show("Voulez-vous vraiment vous déconnecter ?",
        "Déconnexion", MessageBoxButtons.YesNo, MessageBoxIcon.Question);

    if (result == DialogResult.Yes)
    {
        this.Close();
        Application.Restart();
    }
}

1 référence
private void FormAdministrateur_Load(object sender, EventArgs e)
{
}

}
```

Form Gestion Utilisateurs

1. Présentation

Le formulaire FormGestionUtilisateurs permet à l'administrateur de gérer les utilisateurs du système. Il offre des fonctionnalités complètes telles que l'affichage, l'ajout, la modification et la suppression des utilisateurs, tout en assurant la validation des données et la prévention des doublons.

The screenshot shows a software window titled "Gestion des Utilisateurs". The window has a standard Windows-style title bar with minimize, maximize, and close buttons. The main content area is divided into two sections. The top section features a large, empty gray rectangular box, likely intended for displaying a list of users. To the right of this box are four buttons: "Ajouter" (green), "Modifier" (blue), "Supprimer" (red), and "Fermer" (gray). The bottom section is titled "Informations Utilisateur" and contains a form with several input fields: "Nom:", "Prénom:", "Email:", and "Téléphone:" on the left, and "Login:", "Mot de passe:", and "Rôle:" on the right. The "Rôle:" field is a dropdown menu. An "Enregistrer" (green) button is located at the bottom right of the form area.

Form Gestion Utilisateurs

2. Importation des bibliothèques

System.Data : permet de manipuler les données sous forme de tables.

System.Text.RegularExpressions : sert à valider les données avec des expressions régulières (email, téléphone, etc.).

```
using System;
using System.Data;
using System.Text.RegularExpressions;
using Microsoft.Data.SqlClient;
using System.Windows.Forms;
```

3. Déclaration des attributs

Objet utilisé pour établir la connexion avec la base de données

Centralise l'accès SQL dans l'application

Initialise les composants graphiques

Initialise la connexion à la base

Charge les utilisateurs

Initialise les rôles disponibles

4. Constructeur

```
namespace CliniqueVeterinaire
{
    4 références
    public partial class FormGestionUtilisateurs : Form
    {
        private ConnexionBD connexionBD;

        1 référence
        public FormGestionUtilisateurs()
        {
            InitializeComponent();
            connexionBD = new ConnexionBD();
            ChargerUtilisateurs();

            cmbRole.Items.Add("Secrétaire");
            cmbRole.Items.Add("Vétérinaire");

            groupBoxForm.Enabled = false;
        }
    }
}
```

Form Gestion Utilisateurs

5. Chargement des utilisateurs

Exécute une requête SQL :
exclut les administrateurs
trie par nom
Remplit un DataTable
Associe les données au DataGridView

```
private void ChargerUtilisateurs()
{
    using (SqlConnection conn = connexionBD.GetConnection())
    {
        try
        {
            conn.Open();
            string sql = "SELECT Id, Nom, Prenom, Login, Role, Email, Telephone FROM Utilisateur WHERE Role != 'Administrateur' ORDER BY Nom";
            SqlDataAdapter adapter = new SqlDataAdapter(sql, conn);
            DataTable dt = new DataTable();
            adapter.Fill(dt);
            dgvUtilisateurs.DataSource = dt;
            dgvUtilisateurs.AutoSizeColumnsMode = DataGridViewAutoSizeColumnsMode.Fill;

            if (dgvUtilisateurs.Columns["Id"] != null)
                dgvUtilisateurs.Columns["Id"].Visible = false;
        }
        catch (Exception ex)
        {
            MessageBox.Show($"Erreur: {ex.Message}", "Erreur", MessageBoxButtons.OK, MessageBoxIcon.Error);
        }
    }
}
```

6. Vérification des doublons

Rôle :
Vérifie si un login ou email existe déjà
Fonctionnement :
Requête SQL avec COUNT(*)
Exclut l'utilisateur actuel (cas modification)
Retourne true si doublon trouvé

```
private bool UtilisateurExists(string login, string email, int excludeId = 0)
{
    using (SqlConnection conn = connexionBD.GetConnection())
    {
        conn.Open();
        string sql = @"SELECT COUNT(*) FROM Utilisateur
                        WHERE (Login = @Login OR Email = @Email)
                        AND Id != @ExcludeId";
        SqlCommand cmd = new SqlCommand(sql, conn);
        cmd.Parameters.AddWithValue("@Login", login.Trim());
        cmd.Parameters.AddWithValue("@Email", email.Trim());
        cmd.Parameters.AddWithValue("@ExcludeId", excludeId);
        return (int)cmd.ExecuteScalar() > 0;
    }
}
```

Form Gestion Utilisateurs

7. Validation des données

La validation des données permet de vérifier que les informations saisies par l'utilisateur sont correctes avant de les enregistrer dans la base de données. Elle contrôle que tous les champs obligatoires sont remplis et que les valeurs respectent certaines règles, comme le format du nom, du login, de l'email et du numéro de téléphone. Cette étape permet d'éviter les erreurs, les doublons et d'assurer la fiabilité des données enregistrées.

```
private bool IsValidName(string name)
{
    if (string.IsNullOrWhiteSpace(name))
        return false;

    if (!Regex.IsMatch(name.Trim(), @"^[a-zA-ZÀ-ÿ\s\~]+$"))
        return false;

    if (Regex.IsMatch(name.Trim(), @"\d"))
        return false;

    if (name.Trim().Length < 2)
        return false;

    return true;
}

// Validate login (letters only, no spaces, min 3 chars) - REQUIRED
1 référence
private bool IsValidLogin(string login)
{
    if (string.IsNullOrWhiteSpace(login))
        return false;

    if (Regex.IsMatch(login.Trim(), @"\d"))
        return false;

    if (login.Trim().Contains(" "))
        return false;

    if (!Regex.IsMatch(login.Trim(), @"^[a-zA-Z]+$"))
        return false;

    if (login.Trim().Length < 3)
        return false;

    return true;
}
```

```
private bool IsValidPassword(string password)
{
    if (string.IsNullOrWhiteSpace(password))
        return false;

    if (password.Length < 6)
        return false;

    return true;
}

// Validate phone number - REQUIRED (8 digits, starts with 2 or 5)
1 référence
private bool IsValidPhone(string phone)
{
    if (string.IsNullOrWhiteSpace(phone))
        return false;

    if (!Regex.IsMatch(phone.Trim(), @"^\d{8}$"))
        return false;

    if (!phone.Trim().StartsWith("2") && !phone.Trim().StartsWith("5"))
        return false;

    return true;
}

// Validate email format - REQUIRED
1 référence
private bool IsValidEmail(string email)
{
    if (string.IsNullOrWhiteSpace(email))
        return false;

    try
    {
        return Regex.IsMatch(email.Trim(), @"^[^@\s]+@[^\s]+\.[^\s]+$");
    }
}
```

```
private bool ValiderChamps(out string message)
{
    message = "";

    // ===== 1. CHECK EMPTY FIELDS (ALL FIELDS REQUIRED) =====
    string champsVides = "";
    if (string.IsNullOrWhiteSpace(txtNom.Text)) champsVides += "- Nom\n";
    if (string.IsNullOrWhiteSpace(txtPrenom.Text)) champsVides += "- Prénom\n";
    if (string.IsNullOrWhiteSpace(txtLogin.Text)) champsVides += "- Login\n";
    if (string.IsNullOrWhiteSpace(cmbRole.Text)) champsVides += "- Rôle\n";
    if (string.IsNullOrWhiteSpace(txtMotDePasse.Text)) champsVides += "- Mot de passe\n";
    if (string.IsNullOrWhiteSpace(txtEmail.Text)) champsVides += "- Email\n";
    if (string.IsNullOrWhiteSpace(txtTelephone.Text)) champsVides += "- Téléphone\n";

    if (champsVides != "")
    {
        message = "X Les champs suivants sont obligatoires :\n\n{champsVides}";
        return false;
    }

    // ===== 2. VALIDATE NOM =====
    if (!IsValidName(txtNom.Text))
    {
        message = "X Nom invalide.\n\nRègles:\n- Uniquement des lettres\n- Minimum 2 caractères\n- Pas de chiffres";
        return false;
    }

    // ===== 3. VALIDATE PRENOM =====
    if (!IsValidName(txtPrenom.Text))
    {
        message = "X Prénom invalide.\n\nRègles:\n- Uniquement des lettres\n- Minimum 2 caractères\n- Pas de chiffres";
        return false;
    }

    // ===== 4. VALIDATE LOGIN =====
    if (!IsValidLogin(txtLogin.Text))
    {
        message = "X Login invalide.\n\nRègles:\n- Uniquement des lettres\n- Pas d'espaces\n- Pas de chiffres\n- Minimum 3 caractères";
        return false;
    }
}
```

Form Gestion Utilisateurs

8. Bouton Ajouter

Active le formulaire de saisie
Vide les champs
Prépare l'ajout d'un nouvel utilisateur

9. Bouton Modifier

Rôle :
Charge les données de l'utilisateur sélectionné
Fonctionnement :
Vérifie qu'une ligne est sélectionnée
Remplit les champs avec les données existantes

10. Bouton Fermer

Ferme la fenêtre

11. Méthode ClearForm

Réinitialise tous les champs
Prépare une nouvelle saisie

```
private void btnAjouter_Click(object sender, EventArgs e)
{
    groupBoxForm.Enabled = true;
    groupBoxForm.Text = "Ajouter un utilisateur";
    ClearForm();
    txtNom.Focus();
}

1 référence
private void btnModifier_Click(object sender, EventArgs e)
{
    if (dgvUtilisateurs.CurrentRow == null)
    {
        MessageBox.Show("Veuillez sélectionner un utilisateur à modifier",
            "Information", MessageBoxButtons.OK, MessageBoxIcon.Information);
        return;
    }

    groupBoxForm.Enabled = true;
    groupBoxForm.Text = "Modifier un utilisateur";

    DataGridViewRow row = dgvUtilisateurs.CurrentRow;
    txtNom.Text = row.Cells["Nom"].Value?.ToString() ?? "";
    txtPrenom.Text = row.Cells["Prenom"].Value?.ToString() ?? "";
    txtLogin.Text = row.Cells["Login"].Value?.ToString() ?? "";
    cmbRole.Text = row.Cells["Role"].Value?.ToString() ?? "";
    txtEmail.Text = row.Cells["Email"].Value?.ToString() ?? "";
    txtTelephone.Text = row.Cells["Telephone"].Value?.ToString() ?? "";
    txtMotDePasse.Text = "";
}
```

```
private void btnFermer_Click(object sender, EventArgs e)
{
    this.Close();
}

2 références
private void ClearForm()
{
    txtNom.Text = "";
    txtPrenom.Text = "";
    txtLogin.Text = "";
    txtMotDePasse.Text = "";
    cmbRole.SelectedIndex = -1;
    txtEmail.Text = "";
    txtTelephone.Text = "";
}
```

12. Bouton Supprimer

Rôle :

Supprime un utilisateur

Étapes :

Vérifie la sélection , Demande confirmation,

Exécute une requête DELETE, Recharge les données

```
private void btnSupprimer_Click(object sender, EventArgs e)
{
    if (dgvUtilisateurs.CurrentRow == null)
    {
        MessageBox.Show("Veuillez sélectionner un utilisateur à supprimer",
            "Information", MessageBoxButtons.OK, MessageBoxIcon.Information);
        return;
    }

    DialogResult result = MessageBox.Show("Voulez-vous vraiment supprimer cet utilisateur ?",
        "Confirmation", MessageBoxButtons.YesNo, MessageBoxIcon.Question);

    if (result == DialogResult.Yes)
    {
        int id = (int)dgvUtilisateurs.CurrentRow.Cells["Id"].Value;

        using (SqlConnection conn = connexionBD.GetConnection())
        {
            try
            {
                conn.Open();
                string sql = "DELETE FROM Utilisateur WHERE Id = @Id";
                SqlCommand cmd = new SqlCommand(sql, conn);
                cmd.Parameters.AddWithValue("@Id", id);
                cmd.ExecuteNonQuery();

                MessageBox.Show("✓ Utilisateur supprimé avec succès",
                    "Succès", MessageBoxButtons.OK, MessageBoxIcon.Information);
                ChargerUtilisateurs();
            }
            catch (Exception ex)
            {
                MessageBox.Show($"✗ Erreur: {ex.Message}", "Erreur", MessageBoxButtons.OK, MessageBoxIcon.Error);
            }
        }
    }
}
```

13. Bouton Enregistrer

Le bouton Enregistrer permet de sauvegarder un nouvel utilisateur ou de modifier un utilisateur existant dans la base de données. Avant l'enregistrement, il vérifie la validité des données saisies et l'absence de doublons (login ou email). Ensuite, il exécute une requête SQL d'insertion ou de mise à jour selon le cas, puis actualise la liste des utilisateurs et affiche un message de confirmation.

```
private void btnEnregistrer_Click(object sender, EventArgs e)
{
    // ===== VALIDATION =====
    if (!ValiderChamps(out string message))
    {
        MessageBox.Show(message, "Validation des champs", MessageBoxButtons.OK, MessageBoxIcon.Warning);
        return;
    }

    // ===== DUPLICATE CHECK =====
    int currentId = 0;
    if (groupBoxForm.Text == "Modifier un utilisateur" && dgvUtilisateurs.CurrentRow != null)
        currentId = (int)dgvUtilisateurs.CurrentRow.Cells["Id"].Value;

    if (UtilisateurExists(txtLogin.Text.Trim(), txtEmail.Text.Trim(), currentId))
    {
        MessageBox.Show("✗ Un utilisateur avec ce login ou cet email existe déjà.\nVeuillez utiliser des informations différentes.",
            "Doublon détecté", MessageBoxButtons.OK, MessageBoxIcon.Warning);
        txtLogin.Focus();
        return;
    }

    using (SqlConnection conn = connexionBD.GetConnection())
    {
        try
        {
            conn.Open();

            if (groupBoxForm.Text == "Ajouter un utilisateur")
            {
                string sql = @"INSERT INTO Utilisateur (Nom, Prenom, Login, MotDePasse, Role, Email, Telephone)
                VALUES (@Nom, @Prenom, @Login, @MotDePasse, @Role, @Email, @Telephone)";
                SqlCommand cmd = new SqlCommand(sql, conn);
                cmd.Parameters.AddWithValue("@Nom", txtNom.Text.Trim());
                cmd.Parameters.AddWithValue("@Prenom", txtPrenom.Text.Trim());
                cmd.Parameters.AddWithValue("@Login", txtLogin.Text.Trim());
                cmd.Parameters.AddWithValue("@MotDePasse", txtMotDePasse.Text);
                cmd.Parameters.AddWithValue("@Role", cmbRole.Text);
                cmd.Parameters.AddWithValue("@Email", txtEmail.Text.Trim());
                cmd.Parameters.AddWithValue("@Telephone", txtTelephone.Text.Trim());
            }
        }
    }
}
```

Form GestionStock

1. Présentation

Cette interface permet de gérer les médicaments en stock : affichage, ajout, modification, suppression, ainsi que la détection des alertes (stock faible et date d'expiration).

Gestion du Stock - Clinique Vétérinaire

Ajouter

Modifier

Supprimer

Actualiser

Fermer

Informations Médicament

Nom:

Description:

Prix Unitaire (DT):

Fournisseur:

Quantité:

Seuil Alerte:

Date Expiration:

Enregistrer

2. Ajouter

Ce bouton permet de préparer l'ajout d'un nouveau médicament. Lorsqu'on clique dessus, il active le formulaire (`groupBoxForm.Enabled = true`), change le titre en "Ajouter un médicament", vide tous les champs avec `ClearForm()` pour éviter les anciennes données, puis place le curseur dans le champ du nom pour commencer la saisie. Aucun enregistrement n'est encore fait à ce stade.

3.Modifier

Ce bouton sert à modifier un médicament existant. Il vérifie d'abord si une ligne est sélectionnée dans le tableau (`DataGridView`). Si non, il affiche un message. Sinon, il active le formulaire et remplit automatiquement tous les champs avec les données du médicament sélectionné (nom, description, prix, quantité, etc.). Cela permet à l'utilisateur de modifier les valeurs existantes avant de les enregistrer.

4.Supprimer

Ce bouton permet de supprimer un médicament de la base de données. Il vérifie d'abord si un médicament est sélectionné. Ensuite, il demande une confirmation à l'utilisateur. Si l'utilisateur accepte, il récupère l'ID du médicament et exécute une requête SQL `DELETE` pour le supprimer. Après suppression, il recharge la liste des médicaments et affiche un message de succès.

5.Actualiser

Ce bouton sert à mettre à jour l’affichage des médicaments. Il recharge les données depuis la base de données avec `ChargerMedicaments()`, vide le formulaire, désactive le formulaire pour éviter une saisie inutile, et affiche un message pour confirmer que la liste est actualisée. C’est utile après des modifications ou pour rafraîchir les données.

6. Enregistrer

Ce bouton est le plus important : il permet de sauvegarder les données dans la base. Il commence par valider tous les champs avec `ValiderChamps()`. Si tout est correct, il ouvre la connexion à la base de données. Ensuite :
Si on est en mode ajout, il exécute une requête `INSERT` pour ajouter un nouveau médicament.
Si on est en mode modification, il exécute une requête `UPDATE` pour modifier le médicament sélectionné.
Après l’opération, il désactive le formulaire, vide les champs et recharge la liste.

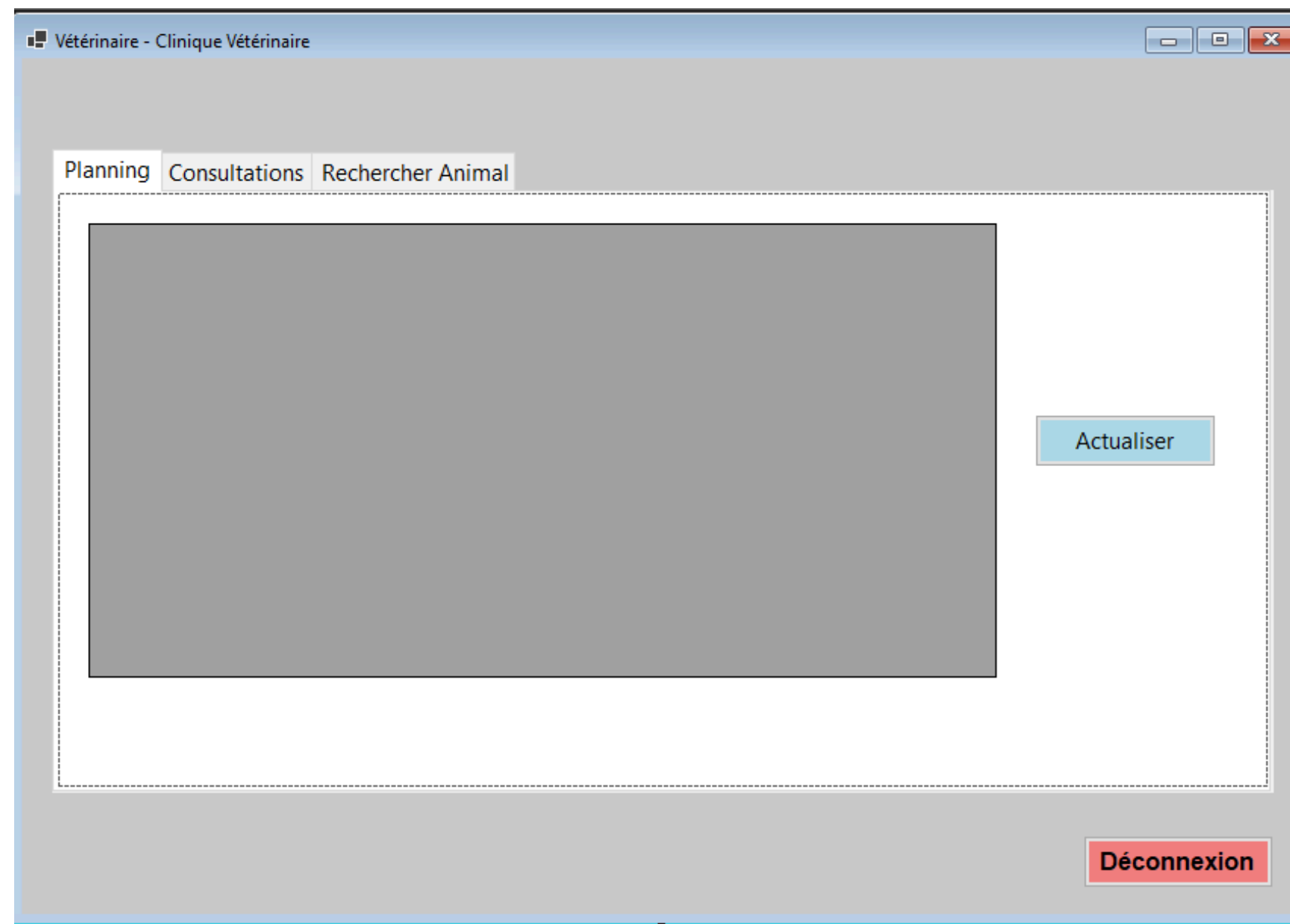
7.Fermer

Ce bouton permet simplement de fermer la fenêtre actuelle (Form). Lorsqu’on clique dessus, l’application ferme cette interface de gestion de stock

Form Veterinaire

1. Présentation

Le formulaire FormVeterinaire représente l'interface principale du vétérinaire dans l'application Clinique Vétérinaire. Il permet de gérer le planning des rendez-vous, d'enregistrer des consultations médicales et de consulter le dossier médical des animaux avec leurs vaccinations.



Form Veterinaire

2. Attributs

connexionBD : permet l'accès à la base de données
nomUtilisateur et prenomUtilisateur : stockent les informations du vétérinaire connecté

Initialise le formulaire

Affiche un message de bienvenue personnalisé

Charge automatiquement le planning et les listes déroulantes

3. Constructeur

3. Validations

Les validations dans le formulaire vétérinaire permettent de garantir la fiabilité et la cohérence des données saisies avant leur enregistrement. Elles vérifient que les champs obligatoires sont bien remplis, que le diagnostic et le traitement contiennent un texte valide (minimum de caractères et non uniquement des chiffres), et que les valeurs médicales comme le poids et la température respectent des formats et des plages acceptables. Le poids doit être un nombre positif inférieur ou égal à 200 kg, tandis que la température doit être comprise entre 30°C et 45°C. Ces contrôles permettent d'éviter les erreurs de saisie et d'assurer la qualité des informations médicales enregistrées.

```
public partial class FormVeterinaire : Form
{
    private ConnexionBD connexionBD;
    private string nomUtilisateur;
    private string prenomUtilisateur;

    1 référence
    public FormVeterinaire(string nom, string prenom)
    {
        InitializeComponent();
        connexionBD = new ConnexionBD();
        nomUtilisateur = nom;
        prenomUtilisateur = prenom;
        lblWelcomeVeto.Text = $"Bienvenue, Dr {prenom} {nom} !";

        ChargerPlanning();
        ChargerComboBoxes();
    }
}
```

```
private bool IsLettersOnly(string text)
{
    return Regex.IsMatch(text.Trim(), @"^[a-zA-ZÀ-ÿ\s\-\_]+$");
}

1 référence
private bool IsValidPoids(string poids, out decimal value)
{
    value = 0;
    if (string.IsNullOrEmpty(poids)) return true;
    if (!decimal.TryParse(poids.Replace(",", "."),
        System.Globalization.NumberStyles.Any,
        System.Globalization.CultureInfo.InvariantCulture, out value))
        return false;
    if (value <= 0 || value > 200) return false;
    return true;
}

1 référence
private bool IsValidTemperature(string temp, out decimal value)
{
    value = 0;
    if (string.IsNullOrEmpty(temp)) return true;
    if (!decimal.TryParse(temp.Replace(",", "."),
        System.Globalization.NumberStyles.Any,
        System.Globalization.CultureInfo.InvariantCulture, out value))
        return false;
    if (value < 30 || value > 45) return false;
    return true;
}
```

Form Veterinaire

4. Chargement des animaux

Rôle :

Remplit la liste des animaux disponibles

Fonctionnement :

Récupère les animaux depuis la base

Ajoute leur Id et Nom dans une ComboBox

Permet de sélectionner un animal pour consultation

```
private void ChargerComboBoxes()
{
    using (SqlConnection conn = connexionBD.GetConnection())
    {
        conn.Open();
        string sql = "SELECT Id, Nom FROM Animal ORDER BY Nom";
        SqlCommand cmd = new SqlCommand(sql, conn);
        SqlDataReader reader = cmd.ExecuteReader();
        cmbAnimalConsult.Items.Clear();
        while (reader.Read())
        {
            cmbAnimalConsult.Items.Add(new KeyValuePair<int, string>(
                (int)reader["Id"], reader["Nom"].ToString()));
        }
        reader.Close();
    }
    cmbAnimalConsult.DisplayMember = "Value";
    cmbAnimalConsult.ValueMember = "Key";
}
```

5. Gestion du planning

Rôle :

Affiche les rendez-vous du jour du vétérinaire

Fonctionnement :

Joint les tables : RendezVous, Animal, Propriétaire

Affiche : animal, propriétaire, date, motif , statut

Bouton actualiser : Recharge le planning Affiche un message de confirmation

```
private void ChargerPlanning()
{
    using (SqlConnection conn = connexionBD.GetConnection())
    {
        try
        {
            conn.Open();
            string sql = @"SELECT r.Id, a.Nom AS Animal, p.Nom + ' ' + p.Prenom AS Proprietaire,
                                r.DateHeure, r.Motif, r.Statut
                        FROM RendezVous r
                        JOIN Animal a ON r.AnimalId = a.Id
                        JOIN Proprietaire p ON a.ProprietaireId = p.Id
                        WHERE r.VeterinaireId = (SELECT Id FROM Utilisateur WHERE Login = 'vet')
                        AND CAST(r.DateHeure AS DATE) = CAST(GETDATE() AS DATE)
                        ORDER BY r.DateHeure";

            SqlDataAdapter adapter = new SqlDataAdapter(sql, conn);
            DataTable dt = new DataTable();
            adapter.Fill(dt);
            dgvPlanning.DataSource = dt;
            dgvPlanning.Columns["Id"].Visible = false;
            dgvPlanning.AutoSizeColumnsMode = DataGridViewAutoSizeColumnsMode.Fill;
        }
        catch (Exception ex)
        {
            MessageBox.Show($"X Erreur: {ex.Message}", "Erreur",
                MessageBoxButtons.OK, MessageBoxIcon.Error);
        }
    }
}

1 référence
private void btnActualiserPlanning_Click(object sender, EventArgs e)
{
    ChargerPlanning();
    MessageBox.Show("☑ Planning actualisé", "Actualisation",
        MessageBoxButtons.OK, MessageBoxIcon.Information);
}
```

Form Veterinaire

6. Module consultations

module Consultation du formulaire vétérinaire permet de gérer les consultations médicales des animaux. Il commence par la sélection d'un animal via une liste déroulante, ce qui permet d'afficher automatiquement ses informations ainsi que son historique médical. Le vétérinaire peut ensuite enregistrer une nouvelle consultation en saisissant le diagnostic, le traitement, le poids, la température et des remarques. Avant l'enregistrement, des validations sont effectuées pour vérifier la présence des champs obligatoires et la validité des données médicales. Une fois la consultation enregistrée, les informations sont insérées dans la base de données et l'historique des consultations est mis à jour pour refléter les nouvelles données.

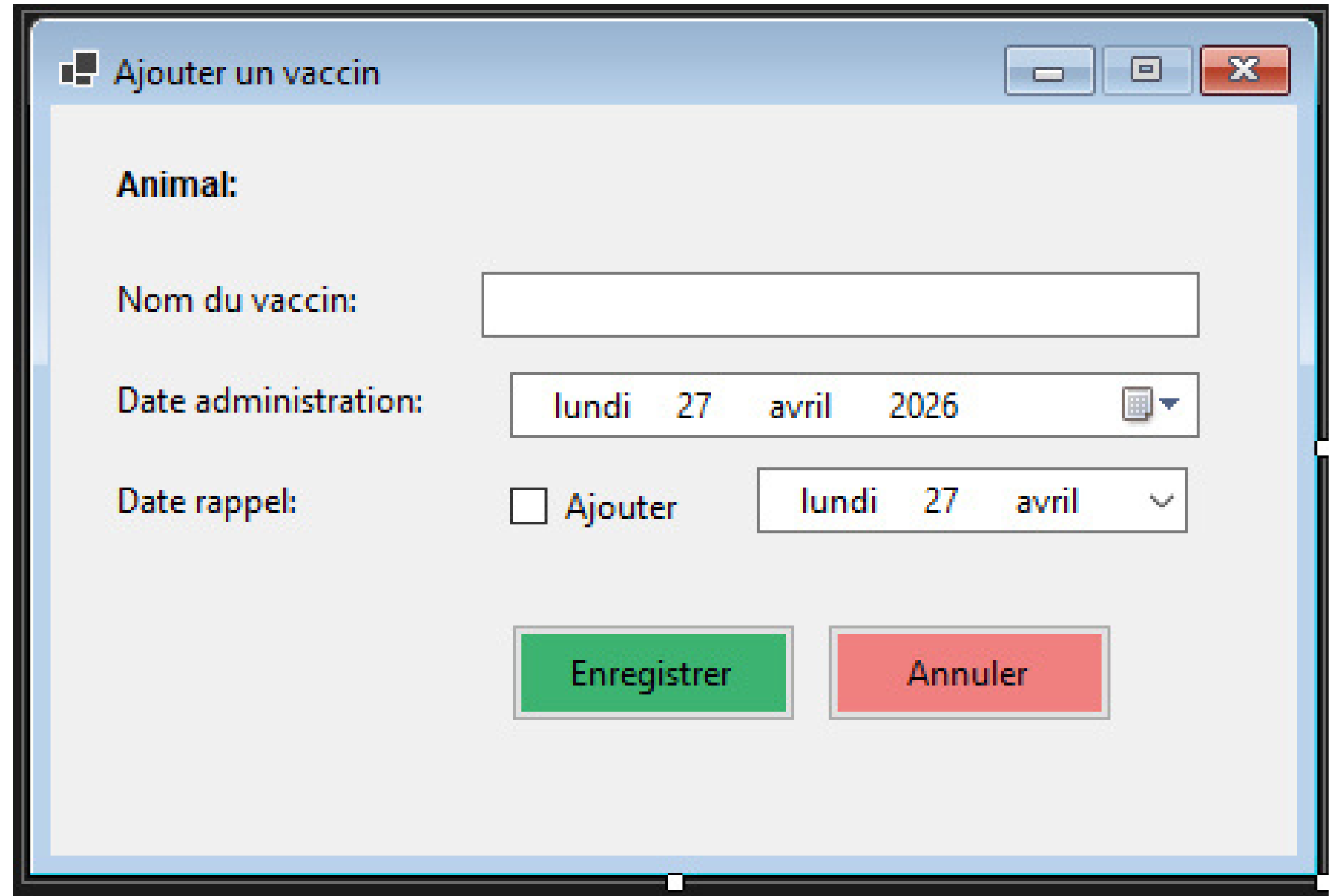
7. Dossier médical

Elle permet au vétérinaire de rechercher un animal par son nom afin d'afficher ses informations complètes ainsi que les données de son propriétaire. Une fois l'animal trouvé, son historique de vaccinations est chargé et affiché dans un tableau. Le vétérinaire peut également ajouter un nouveau vaccin via un formulaire dédié, puis la liste est automatiquement mise à jour. Cette partie permet ainsi un suivi médical complet et centralisé de chaque animal.

Form Ajouter Vaccin

1. Présentation

Cette interface permet d'ajouter facilement un vaccin pour un animal dans la clinique vétérinaire. Elle affiche le nom de l'animal concerné et permet de saisir les informations essentielles comme le nom du vaccin, la date d'administration et éventuellement une date de rappel. Grâce aux validations intégrées, elle assure la fiabilité des données saisies. L'interface est simple, rapide à utiliser et facilite le suivi médical des animaux.



The screenshot shows a software window titled "Ajouter un vaccin". The window has a standard Windows-style title bar with minimize, maximize, and close buttons. The main content area is light gray and contains the following elements:

- Animal:** A label followed by a large, empty text input field.
- Nom du vaccin:** A label followed by a text input field.
- Date administration:** A label followed by a date picker showing "lundi 27 avril 2026" and a calendar icon.
- Date rappel:** A label followed by a checkbox labeled "Ajouter" and a date picker showing "lundi 27 avril".
- Buttons:** Two buttons at the bottom: a green "Enregistrer" button and a red "Annuler" button.

2.Enregistrer

Ce bouton permet d'ajouter un vaccin dans la base de données pour un animal. Lorsqu'on clique dessus, le système vérifie d'abord que le nom du vaccin est valide (au moins 2 caractères), que la date d'administration n'est pas dans le futur, et que la date de rappel (si activée) est bien après la date d'administration. Si toutes les validations sont correctes, il enregistre les informations dans la table Vaccination avec l'ID de l'animal et du vétérinaire. Ensuite, un message de succès s'affiche, la fenêtre se ferme et renvoie un résultat "OK".

```
using (SqlConnection conn = connexionBD.GetConnection())
{
    try
    {
        conn.Open();
        string sql = @"INSERT INTO Vaccination (NomVaccin, DateAdministration, DateRappel, AnimalId, VeterinaireId)
            VALUES (@NomVaccin, @DateAdmin, @DateRappel, @AnimalId,
            (SELECT Id FROM Utilisateur WHERE Login = 'vet'))";
        SqlCommand cmd = new SqlCommand(sql, conn);
        cmd.Parameters.AddWithValue("@NomVaccin", txtNomVaccin.Text.Trim());
        cmd.Parameters.AddWithValue("@DateAdmin", dtpDateAdmin.Value);
        cmd.Parameters.AddWithValue("@DateRappel", dateRappel.HasValue ? (object)dateRappel.Value : DBNull.Value);
        cmd.Parameters.AddWithValue("@AnimalId", animalId);
        cmd.ExecuteNonQuery();

        MessageBox.Show("☑ Vaccin ajouté avec succès", "Succès", MessageBoxButtons.OK, MessageBoxIcon.Information);
        this.DialogResult = DialogResult.OK; // ← ICI le changement
        this.Close();
    }
    catch (Exception ex)
    {
        MessageBox.Show($"✗ Erreur: {ex.Message}", "Erreur", MessageBoxButtons.OK, MessageBoxIcon.Error);
    }
}
```

```
private void btnEnregistrer_Click(object sender, EventArgs e)
{
    if (!IsValidVaccinName(txtNomVaccin.Text))
    {
        MessageBox.Show("✗ Nom du vaccin invalide.\n\nRègles:\n- Minimum 2 caractères",
            "Validation", MessageBoxButtons.OK, MessageBoxIcon.Warning);
        txtNomVaccin.Focus();
        return;
    }

    if (!IsValidAdminDate(dtpDateAdmin.Value))
    {
        MessageBox.Show("✗ La date d'administration ne peut pas être dans le futur.",
            "Validation", MessageBoxButtons.OK, MessageBoxIcon.Warning);
        dtpDateAdmin.Focus();
        return;
    }

    DateTime? dateRappel = null;
    if (chkRappel.Checked)
    {
        if (!IsValidRappelDate(dtpDateAdmin.Value, dtpDateRappel.Value))
        {
            MessageBox.Show("✗ La date de rappel doit être postérieure à la date d'administration.",
                "Validation", MessageBoxButtons.OK, MessageBoxIcon.Warning);
            dtpDateRappel.Focus();
            return;
        }
        dateRappel = dtpDateRappel.Value;
    }
}
```

3. Annuler

Ce bouton permet d'annuler l'opération d'ajout du vaccin. Lorsqu'on clique dessus, aucune donnée n'est enregistrée. La fenêtre se ferme simplement et renvoie un résultat "Cancel", ce qui indique que l'utilisateur a abandonné l'action.

```
private void btnAnnuler_Click(object sender, EventArgs e)
{
    this.DialogResult = DialogResult.Cancel; // ← ICI le changement
    this.Close();
}
```

4. Case à cocher Rappel (chkRappel_CheckedChanged)

Même si ce n'est pas un bouton, cette case permet de gérer l'activation de la date de rappel. Lorsqu'elle est cochée, le champ de date de rappel devient actif pour permettre la saisie. Lorsqu'elle est décochée, ce champ est désactivé et une date par défaut (6 mois après la date d'administration) est automatiquement proposée.

```
private void chkRappel_CheckedChanged(object sender, EventArgs e)
{
    dtpDateRappel.Enabled = chkRappel.Checked;
    if (!chkRappel.Checked)
    {
        dtpDateRappel.Value = dtpDateAdmin.Value.AddMonths(6);
    }
}
```

Form Secrétaire

1. Présentation

Le Formulaire Secrétaire constitue le module principal de gestion administrative de la clinique vétérinaire. Il permet de gérer les propriétaires, les animaux et les rendez-vous de manière structurée. Grâce à une interface intuitive, la secrétaire peut ajouter, modifier et supprimer les données tout en assurant leur cohérence à travers des validations strictes. Ce module facilite également la planification des consultations et le suivi des informations essentielles, garantissant ainsi une organisation efficace du travail au sein de la clinique.

The screenshot displays the 'Form Secrétaire' interface. At the top, there are three tabs: 'Propriétaires', 'Animaux', and 'Rendez-vous'. The 'Propriétaires' tab is selected. Below the tabs is a large, empty rectangular area, likely a table for listing owners. To the right of this area are four buttons: 'Ajouter' (green), 'Modifier' (blue), 'Supprimer' (red), and 'Actualiser' (light blue). Below this area is a section titled 'Informations Propriétaire' containing a form with six input fields: 'Nom:', 'Prénom:', 'Téléphone:', 'Email:', 'Adresse:', and 'Ville:'. An 'Enregistrer' button (green) is located at the bottom right of this form. At the very bottom of the interface are two buttons: 'Factures' (light blue) and 'Déconnexion' (red).

Form Secrétaire

1. Validation des données

Les validations vérifient que les champs obligatoires ne sont pas vides, que les noms contiennent uniquement des lettres avec un minimum de caractères, que le téléphone respecte 8 chiffres et commence par 2 ou 5, et que l'email est au bon format. Pour les animaux, le sexe doit être M ou F, la date de naissance ne doit pas être dans le futur, et les champs comme espèce, race et couleur respectent des règles de format. Pour les rendez-vous, le motif doit contenir au moins 3 caractères et la date ne peut pas être dans le passé. Ces validations garantissent la cohérence et la qualité des données avant l'enregistrement.

```
private bool IsValidName(string name)
{
    if (string.IsNullOrEmpty(name))
        return false;

    if (!Regex.IsMatch(name.Trim(), @"^[a-zA-ZÄ-ÿ\s\~]+$"))
        return false;

    if (Regex.IsMatch(name.Trim(), @"\d"))
        return false;

    if (name.Trim().Length < 2)
        return false;

    return true;
}

// Validate phone number (8 digits, starts with 2 or 5) - REQUIRED
1 référence
private bool IsValidPhone(string phone)
{
    if (string.IsNullOrEmpty(phone))
        return false;

    if (!Regex.IsMatch(phone.Trim(), @"^\d{8}$"))
        return false;

    if (!phone.Trim().StartsWith("2") && !phone.Trim().StartsWith("5"))
        return false;

    return true;
}

// Validate email format - REQUIRED
1 référence
private bool IsValidEmail(string email)
```

```
private bool ValiderProprietaire(out string message)
{
    message = "";

    string champsVides = "";
    if (string.IsNullOrEmpty(txtNomProp.Text)) champsVides += "- Nom\n";
    if (string.IsNullOrEmpty(txtPrenomProp.Text)) champsVides += "- Prénom\n";
    if (string.IsNullOrEmpty(txtTelProp.Text)) champsVides += "- Téléphone\n";
    if (string.IsNullOrEmpty(txtEmailProp.Text)) champsVides += "- Email\n";

    if (champsVides != "")
    {
        message = $"X Les champs suivants sont obligatoires :\n\n{champsVides}";
        return false;
    }

    if (!IsValidName(txtNomProp.Text))
    {
        message = "X Nom invalide.\n\nRègles:\n- Uniquement des lettres\n- Minimum 2 caractères";
        return false;
    }

    if (!IsValidName(txtPrenomProp.Text))
    {
        message = "X Prénom invalide.\n\nRègles:\n- Uniquement des lettres\n- Minimum 2 caractères";
        return false;
    }

    if (!IsValidPhone(txtTelProp.Text))
    {
        message = "X Numéro de téléphone invalide.\n\nRègles:\n- 8 chiffres exactement\n- Doit commencer par 2 ou 5";
        return false;
    }

    if (!IsValidEmail(txtEmailProp.Text))
    {
        message = "X Email invalide.\n\nRègles:\n- Doit contenir '@'\n- Doit contenir '.'\n- Extension obligatoire";
        return false;
    }

    return true;
}
```

Form Secrétaire

2.Module Propriétaire

Cette partie permet d'ajouter, modifier, supprimer et afficher les propriétaires. Elle utilise une validation complète (nom, prénom, téléphone, email, etc.) avant l'enregistrement. Elle vérifie aussi l'existence des doublons (téléphone ou email déjà utilisés). Les données sont enregistrées ou mises à jour dans la base SQL, puis les listes et combobox sont automatiquement actualisées.

```
private bool ValiderProprietaire(out string message)
{
    message = "";

    string champsVides = "";
    if (string.IsNullOrEmpty(txtNomProp.Text)) champsVides += "- Nom\n";
    if (string.IsNullOrEmpty(txtPrenomProp.Text)) champsVides += "- Prénom\n";
    if (string.IsNullOrEmpty(txtTelProp.Text)) champsVides += "- Téléphone\n";
    if (string.IsNullOrEmpty(txtEmailProp.Text)) champsVides += "- Email\n";

    if (champsVides != "")
    {
        message = $"X Les champs suivants sont obligatoires :\n\n{champsVides}";
        return false;
    }

    if (!IsValidName(txtNomProp.Text))
    {
        message = "X Nom invalide.\n\nRègles:\n- Uniquement des lettres\n- Minimum 2 caractères\n- Pas de chiffres";
        return false;
    }

    if (!IsValidName(txtPrenomProp.Text))
    {
        message = "X Prénom invalide.\n\nRègles:\n- Uniquement des lettres\n- Minimum 2 caractères\n- Pas de chiffres";
        return false;
    }

    if (!IsValidPhone(txtTelProp.Text))
    {
        message = "X Numéro de téléphone invalide.\n\nRègles:\n- 8 chiffres exactement\n- Doit commencer par 2 ou 5\n- Exemples: 23*****, 51*****";
        return false;
    }

    if (!IsValidEmail(txtEmailProp.Text))
    {
        message = "X Email invalide.\n\nRègles:\n- Doit contenir '@'\n- Doit contenir '.'\n- Exemple: nom@domaine.tn";
        return false;
    }
}
```

```
private bool ProprietaireExists(string telephone, string email, int excludeId = 0)
{
    using (SqlConnection conn = connexionBD.GetConnection())
    {
        conn.Open();
        string sql = @"SELECT COUNT(*) FROM Proprietaire
                        WHERE (Telephone = @Tel OR Email = @Email)
                        AND Id != @ExcludeId";
        SqlCommand cmd = new SqlCommand(sql, conn);
        cmd.Parameters.AddWithValue("@Tel", telephone.Trim());
        cmd.Parameters.AddWithValue("@Email", email.Trim());
        cmd.Parameters.AddWithValue("@ExcludeId", excludeId);
        return (int)cmd.ExecuteScalar() > 0;
    }
}
```

Form Secrétaire

3.Module Animal

Ce module gère les animaux liés aux propriétaires. Il permet l'ajout, la modification et la suppression. Chaque animal est validé (nom, espèce, sexe, date de naissance, etc.) avant insertion. Le système empêche les doublons pour un même propriétaire. Les données sont affichées dans un DataGridView et les listes sont mises à jour automatiquement.

```
private void ChargerAnimaux()
{
    using (SqlConnection conn = connexionBD.GetConnection())
    {
        try
        {
            conn.Open();
            string sql = @"SELECT a.Id, a.Nom, a.Espece, a.Race, a.Sexe, a.DateNaissance, a.Couleur,
                             p.Nom + ' ' + p.Prenom AS Proprietaire
                           FROM Animal a
                           JOIN Proprietaire p ON a.ProprietaireId = p.Id
                           ORDER BY a.Nom";
            SqlDataAdapter adapter = new SqlDataAdapter(sql, conn);
            DataTable dt = new DataTable();
            adapter.Fill(dt);
            dgvAnimaux.DataSource = dt;
            dgvAnimaux.Columns["Id"].Visible = false;
        }
        catch (Exception ex)
        {
            MessageBox.Show($"Erreur: {ex.Message}", "Erreur", MessageBoxButtons.OK, MessageBoxIcon.Error);
        }
    }
}

1 référence
private void btnAjouterAnimal_Click(object sender, EventArgs e)
{
    groupBoxAnimal.Enabled = true;
    groupBoxAnimal.Text = "Ajouter un animal";
    ClearAnimalForm();
    txtNomAnimal.Focus();
}
```

```
private void btnModifierAnimal_Click(object sender, EventArgs e)
{
    if (dgvAnimaux.CurrentRow == null)
    {
        MessageBox.Show("Sélectionnez un animal", "Information", MessageBoxButtons.OK, MessageBoxIcon.Information);
        return;
    }
    groupBoxAnimal.Enabled = true;
    groupBoxAnimal.Text = "Modifier un animal";
    DataGridViewRow row = dgvAnimaux.CurrentRow;
    txtNomAnimal.Text = row.Cells["Nom"].Value?.ToString() ?? "";
    cmbEspece.Text = row.Cells["Espece"].Value?.ToString() ?? "";
    txtRace.Text = row.Cells["Race"].Value?.ToString() ?? "";
    cmbSexe.Text = row.Cells["Sexe"].Value?.ToString() ?? "";
    dtpDateNaiss.Value = row.Cells["DateNaissance"].Value != DBNull.Value ? (DateTime)row.Cells["DateNaissance"].Value : DateTime.Now;
    txtCouleur.Text = row.Cells["Couleur"].Value?.ToString() ?? "";

    string proprietaire = row.Cells["Proprietaire"].Value?.ToString() ?? "";
    for (int i = 0; i < cmbProprietaire.Items.Count; i++)
    {
        if (cmbProprietaire.GetItemText(cmbProprietaire.Items[i]) == proprietaire)
        {
            cmbProprietaire.SelectedIndex = i;
            break;
        }
    }
}
```

Form Secrétaire

4. Module Rendez-vous

Ce module permet de planifier, modifier et annuler les rendez-vous. Les validations assurent que l'animal, le vétérinaire et le motif sont correctement renseignés et que la date est valide. Il existe aussi une vérification des doublons pour éviter deux rendez-vous identiques au même moment. Les rendez-vous sont affichés dans un tableau et peuvent être mis à jour en temps réel.

```
private void btnAjouterRDV_Click(object sender, EventArgs e)
{
    groupBoxRDV.Enabled = true;
    groupBoxRDV.Text = "Ajouter un rendez-vous";
    ClearRDVForm();
    cmbAnimalRDV.Focus();
}

1 référence
private void btnModifieurRDV_Click(object sender, EventArgs e)
{
    if (dgvRendezVous.CurrentRow == null)
    {
        MessageBox.Show("Sélectionnez un rendez-vous", "Information", MessageBoxButtons.OK, MessageBoxIcon.Information);
        return;
    }
    groupBoxRDV.Enabled = true;
    groupBoxRDV.Text = "Modifier un rendez-vous";

    DataGridViewRow row = dgvRendezVous.CurrentRow;
    string animal = row.Cells["Animal"].Value?.ToString() ?? "";
    string veterinaire = row.Cells["Veterinaire"].Value?.ToString() ?? "";

    for (int i = 0; i < cmbAnimalRDV.Items.Count; i++)
    {
        if (cmbAnimalRDV.GetItemText(cmbAnimalRDV.Items[i]) == animal)
        {
            cmbAnimalRDV.SelectedIndex = i;
            break;
        }
    }

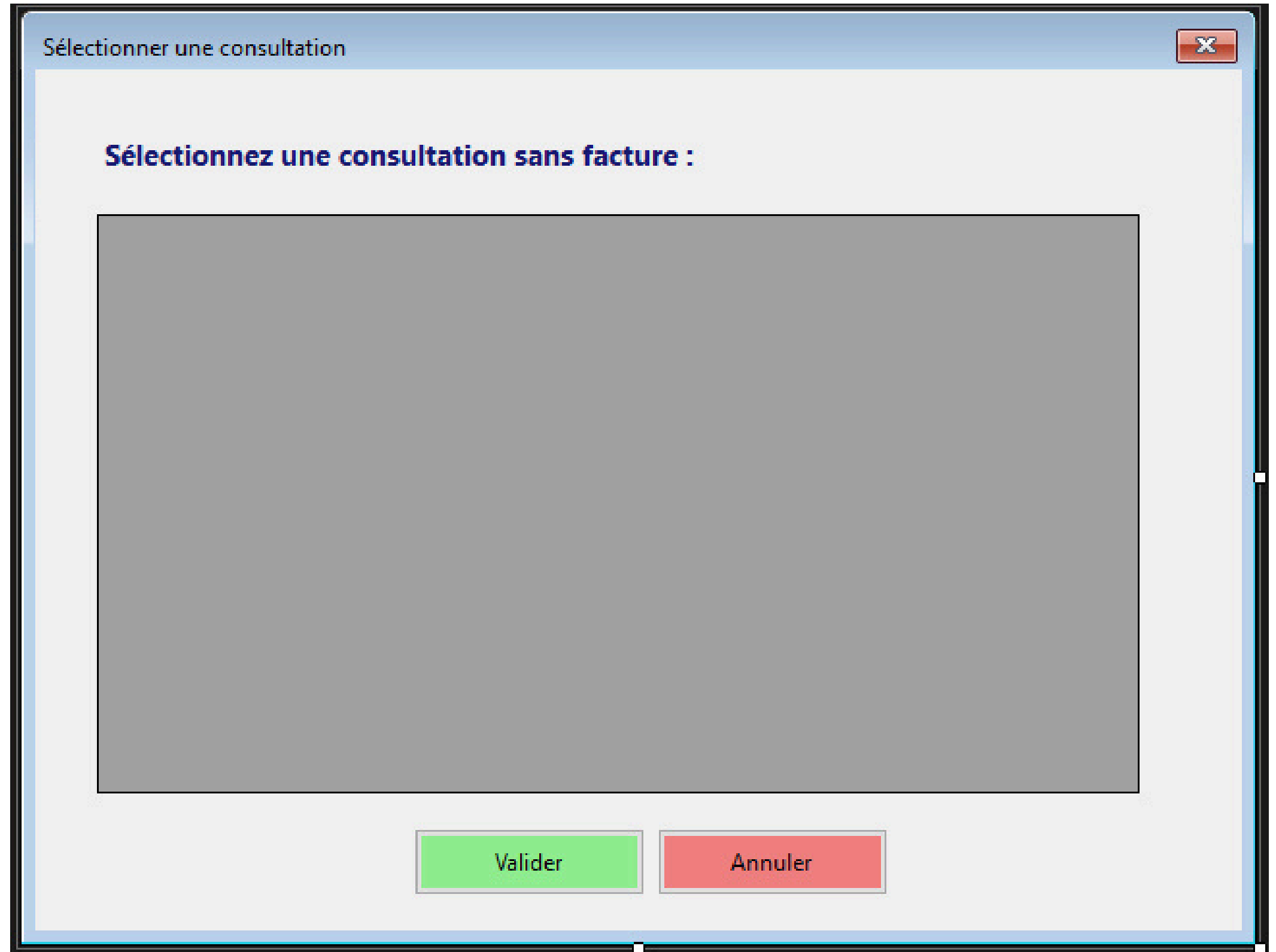
    for (int i = 0; i < cmbVetoRDV.Items.Count; i++)
    {
        if (cmbVetoRDV.GetItemText(cmbVetoRDV.Items[i]) == veterinaire)
        {
            cmbVetoRDV.SelectedIndex = i;
            break;
        }
    }
}
```

```
private void btnAnnulerRDV_Click(object sender, EventArgs e)
{
    if (dgvRendezVous.CurrentRow == null)
    {
        MessageBox.Show("Sélectionnez un rendez-vous", "Information", MessageBoxButtons.OK, MessageBoxIcon.Information);
        return;
    }
    DialogResult result = MessageBox.Show("Annuler ce rendez-vous ?", "Confirmation", MessageBoxButtons.YesNo, MessageBoxIcon.Question);
    if (result == DialogResult.Yes)
    {
        int id = (int)dgvRendezVous.CurrentRow.Cells["Id"].Value;
        using (SqlConnection conn = connexionBD.GetConnection())
        {
            try
            {
                conn.Open();
                string sql = "UPDATE RendezVous SET Statut = 'Annule' WHERE Id = @Id";
                SqlCommand cmd = new SqlCommand(sql, conn);
                cmd.Parameters.AddWithValue("@Id", id);
                cmd.ExecuteNonQuery();
                MessageBox.Show("✅ Rendez-vous annulé", "Succès", MessageBoxButtons.OK, MessageBoxIcon.Information);
                ChargerRendezVous();
            }
            catch (Exception ex)
            {
                MessageBox.Show($"❌ Erreur: {ex.Message}", "Erreur", MessageBoxButtons.OK, MessageBoxIcon.Error);
            }
        }
    }
}
```

Form ChoisirConsultation

1. Présentation

L'interface FormChoisirConsultation permet à l'utilisateur (le vétérinaire ou le personnel) de sélectionner une consultation qui n'a pas encore été facturée. Elle contient principalement un tableau (DataGridView) qui affiche la liste des consultations avec des informations comme le nom de l'animal, le propriétaire, la date et le diagnostic. En haut, un label indique à l'utilisateur ce qu'il doit faire. En bas, il y a deux boutons : Valider pour confirmer le choix et continuer, et Annuler pour quitter sans rien faire.



The screenshot shows a Windows-style dialog box titled "Sélectionner une consultation". Inside the dialog, there is a label "Sélectionnez une consultation sans facture :" in blue text. Below the label is a large, empty gray rectangular area, which represents the DataGridView for displaying consultations. At the bottom of the dialog, there are two buttons: a green button labeled "Valider" and a red button labeled "Annuler". The dialog box has a standard Windows title bar with a close button (X) in the top right corner.

2. Valider

Lorsque l'utilisateur clique sur le bouton Valider, le programme vérifie d'abord si une ligne (consultation) est sélectionnée dans le tableau. Si aucune ligne n'est sélectionnée, un message s'affiche pour demander à l'utilisateur de choisir une consultation. Si une consultation est bien sélectionnée, le programme récupère son identifiant (ConsultationId) ainsi que le nom de l'animal. Ensuite, il se connecte à la base de données pour obtenir l'AnimalId correspondant à cette consultation. Après cela, une nouvelle fenêtre (FormMontantFacture) s'ouvre pour demander à l'utilisateur de saisir le montant de la facture. Si l'utilisateur valide ce montant, celui-ci est enregistré (MontantHT), puis le formulaire se ferme en confirmant l'opération.

```
private void btnValider_Click(object sender, EventArgs e)
{
    if (dgvConsultations.CurrentRow == null)
    {
        MessageBox.Show("Sélectionnez une consultation", "Information",
            MessageBoxButtons.OK, MessageBoxIcon.Warning);
        return;
    }

    ConsultationId = (int)dgvConsultations.CurrentRow.Cells["Id"].Value;
    AnimalNom = dgvConsultations.CurrentRow.Cells["Animal"].Value?.ToString() ?? "";

    using (SqlConnection conn = connexionBD.GetConnection())
    {
        conn.Open();
        string sql = "SELECT AnimalId FROM Consultation WHERE Id = @Id";
        SqlCommand cmd = new SqlCommand(sql, conn);
        cmd.Parameters.AddWithValue("@Id", ConsultationId);
        AnimalId = (int)cmd.ExecuteScalar();
    }

    // Demander le montant
    FormMontantFacture formMontant = new FormMontantFacture();
    if (formMontant.ShowDialog() == DialogResult.OK)
    {
        MontantHT = formMontant.MontantHT;
        DialogResult = DialogResult.OK;
        this.Close();
    }
}
```

3. Annuler

Lorsque l'utilisateur clique sur le bouton Annuler, aucune action n'est effectuée sur les données. Le formulaire se ferme simplement et renvoie un résultat indiquant que l'opération a été annulée (DialogResult.Cancel). Cela permet de revenir à l'écran précédent sans sélectionner de consultation ni créer de facture.

```
private void btnAnnuler_Click_1(object sender, EventArgs e)
{
    DialogResult = DialogResult.Cancel;
    this.Close();
}
```

Form GestionFactures

1. Présentation

Cette interface permet de gérer les factures de la clinique vétérinaire : création automatique ou manuelle, calcul du montant TTC, suivi du paiement et mise à jour des données. Elle offre aussi la possibilité de marquer une facture comme payée et de générer un PDF professionnel. Elle facilite ainsi l'organisation et le suivi financier.

The screenshot shows a web application window titled "Gestion des Factures - Clinique Vétérinaire". The interface is divided into two main sections. The top section features a large, empty gray rectangular area on the left, likely for displaying a generated invoice. To the right of this area is a vertical column of five buttons: "Générer Facture" (green), "Marquer Payée" (blue), "Actualiser" (light blue), "Imprimer" (light gray), and "Fermer" (red). The bottom section, titled "Détails de la facture", contains several input fields and a checkbox. It includes fields for "Numéro facture:", "Date:" (with a calendar icon), "Montant HT (DT):", "TVA (%):" (containing the value "20"), "Montant TTC (DT):", "Mode" (a dropdown menu), and a checkbox labeled "Payée". A green "Enregistrer" button is positioned at the bottom right of this section.

2.Générer Facture

Ce bouton permet de créer une facture à partir d'une consultation existante. Lorsqu'on clique dessus, une fenêtre s'ouvre pour choisir une consultation. Si l'utilisateur valide, les informations liées (animal, montant HT, etc.) sont récupérées automatiquement. Le formulaire est activé, un numéro de facture unique est généré avec la date et l'heure, la date est initialisée à aujourd'hui, et la TVA est fixée par défaut à 20 %. Le montant TTC est ensuite calculé automatiquement.

3.Marquer Payée

Ce bouton permet de changer le statut d'une facture en "payée". Il vérifie d'abord qu'une facture est sélectionnée. Si elle est déjà payée, il affiche un message. Sinon, il demande confirmation à l'utilisateur, puis met à jour la base de données avec une requête UPDATE en définissant Paye = 1 et en enregistrant la date de paiement. Enfin, il recharge la liste des factures pour afficher le nouveau statut

4.Actualiser

Ce bouton sert à rafraîchir la liste des factures affichées. Il recharge les données depuis la base avec `ChargerFactures()` et affiche un message pour informer l'utilisateur que la liste a été mise à jour. C'est utile après une modification ou pour voir les dernières données.

5.Imprimer

Ce bouton permet de générer un fichier PDF de la facture sélectionnée. Il récupère toutes les informations de la facture (numéro, date, montants, client, animal, diagnostic...). Ensuite, il demande à l'utilisateur où enregistrer le fichier via une fenêtre de sauvegarde. Il utilise la bibliothèque iTextSharp pour créer un PDF structuré avec titre, informations client, tableau des montants et total à payer. Une fois terminé, il enregistre le fichier et affiche un message de succès avec l'emplacement.

6.Fermer

Ce bouton permet simplement de fermer la fenêtre de gestion des factures. Il appelle `this.Close()` pour quitter l'interface actuelle.

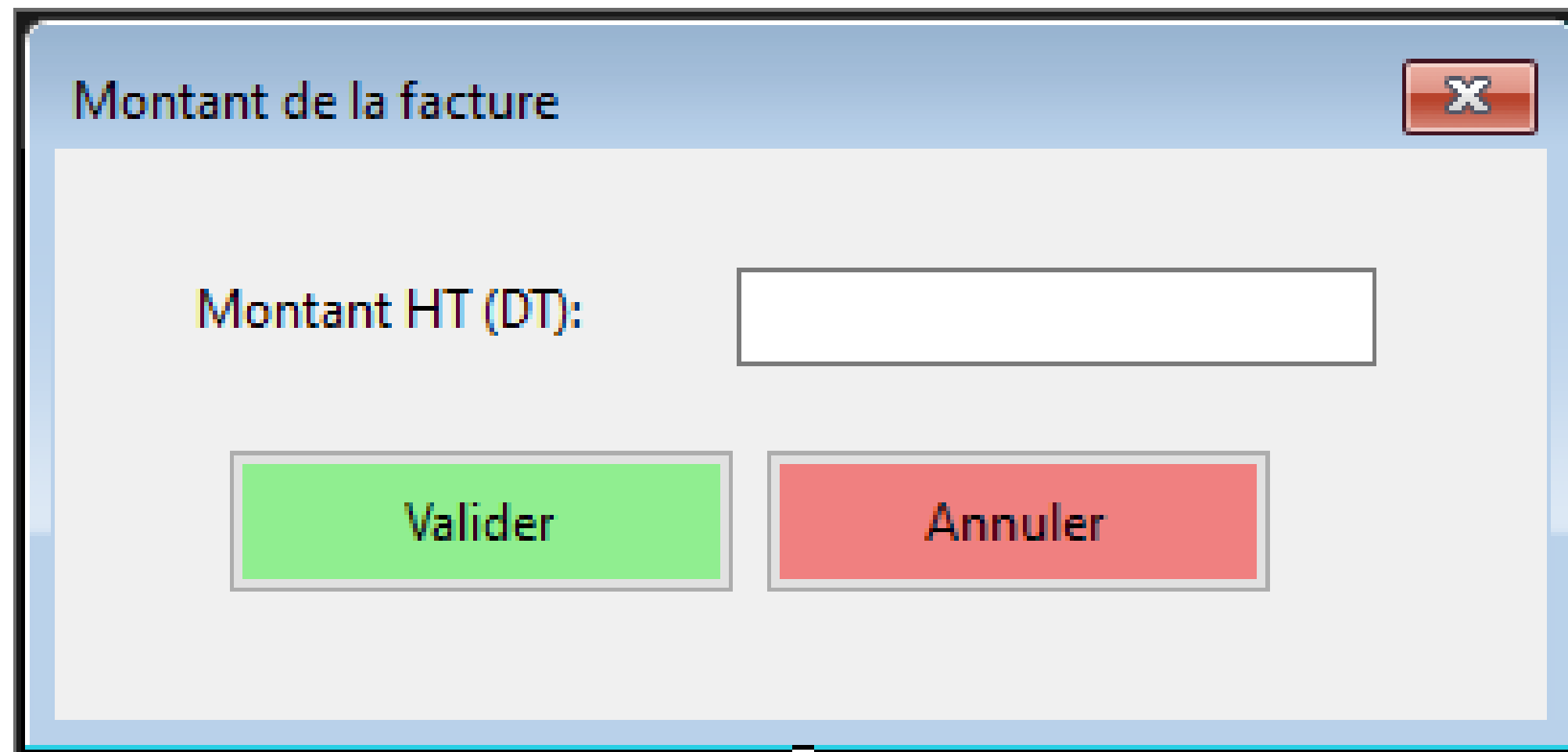
7.Enregister Facture

Ce bouton sert à enregistrer la facture dans la base de données. Il commence par vérifier que tous les champs obligatoires sont correctement remplis (numéro, montant HT, TVA, etc.). Si la facture est marquée comme payée, il exige aussi un mode de paiement. Ensuite, il exécute une requête SQL INSERT pour sauvegarder toutes les informations (montants, statut, consultation, animal...). Après l'enregistrement, il réinitialise le formulaire, désactive la zone de saisie et recharge la liste des factures.

Form Montant Facture

1. Présentation

Le formulaire FormMontantFacture permet de saisir le montant hors taxe (HT) d'une facture dans l'application. Il assure la validation de la valeur saisie avant de l'enregistrer et de la transmettre aux autres parties du programme.



The image shows a screenshot of a software dialog box titled "Montant de la facture". The dialog box has a light blue header bar with the title and a red close button (X) in the top right corner. The main area has a light gray background. It contains a label "Montant HT (DT):" followed by a white text input field. Below the input field, there are two buttons: a green button labeled "Valider" and a red button labeled "Annuler".

Form Montant Facture

2. Attribut principal

Stocke le montant HT saisi par l'utilisateur
Accessible en lecture depuis d'autres formulaires
Protégé en écriture pour éviter les modifications externes

Rôle :

Initialise le formulaire

Configure l'apparence du label

Détails :

Affiche le texte "Montant HT (DT)"

Applique une police en gras

Change la couleur en bleu foncé pour améliorer l'interface

3. Constructeur

Rôle :

Vérifie et valide le montant saisi

Fonctionnement :

Convertit le texte en nombre (decimal)

Vérifie que le montant est supérieur à 0

Si invalide → message d'erreur

Si valide → stocke la valeur et ferme le formulaire avec succès

4. Bouton Valider

```
namespace CliniqueVeterinaire
{
    4 références
    public partial class FormMontantFacture : Form
    {
        2 références
        public decimal MontantHT { get; private set; }

        1 référence
        public FormMontantFacture()
        {
            InitializeComponent();

            // Configurer le label
            lblMontant.Text = "💰 Montant HT (DT):";
            lblMontant.Font = new Font("Arial", 9, FontStyle.Bold);
            lblMontant.ForeColor = Color.DarkBlue;
        }
    }
}
```

```
private void btnValider_Click_1(object sender, EventArgs e)
{
    if (!decimal.TryParse(txtMontant.Text, out decimal montant) || montant <= 0)
    {
        MessageBox.Show("Veuillez saisir un montant valide (supérieur à 0)",
            "Montant invalide", MessageBoxButtons.OK, MessageBoxIcon.Warning);
        txtMontant.Focus();
        return;
    }

    MontantHT = montant;
    DialogResult = DialogResult.OK;
    this.Close();
}

1 référence
private void btnAnnuler_Click_1(object sender, EventArgs e)
{
    DialogResult = DialogResult.Cancel;
    this.Close();
}
```

Form Montant Facture

5. Bouton Annuler

Rôle :

Ferme le formulaire sans enregistrer de montant

Retourne un résultat "Annulé"

6. Contrôle de saisie

Rôle :

Empêche la saisie de caractères non numériques

Fonctionnement :

Autorise seulement : chiffres, virgule, point, touches de contrôle (backspace)

Empêche la saisie de plusieurs séparateurs décimaux

```
private void txtMontant_KeyPress(object sender, KeyPressEventArgs e)
{
    // Permettre seulement les chiffres, virgule et point
    if (!char.IsControl(e.KeyChar) && !char.IsDigit(e.KeyChar) &&
        e.KeyChar != ',' && e.KeyChar != '.')
    {
        e.Handled = true;
    }

    // Permettre un seul séparateur décimal
    if ((e.KeyChar == ',' || e.KeyChar == '.') &&
        (txtMontant.Text.Contains(",") || txtMontant.Text.Contains(".")))
    {
        e.Handled = true;
    }
}
```